Aspect-Oriented Programming (AOP)

Spring Framework — introductory notes based on the supplied lecture transcript and diagrams.

1. What Is AOP?

Aspect-Oriented Programming (AOP) is a Spring module used to separate cross-cutting concerns from the main business logic. The lecture compares AOP with filters and interceptors: a filter wraps a raw incoming request, an interceptor wraps a controller/handler, and AOP can apply supporting logic around methods across service classes.

Quick distinction

Filters — work around the raw HTTP request/response at the servlet layer.
Interceptors — work around a mapped Spring MVC handler/controller.
AOP — can wrap method execution in Spring-managed application classes, such as service methods.

2. Business Logic vs Infrastructure Logic

Business logic represents what the application does for its domain: creating students, managing payments, or placing orders. This logic changes according to the application's domain.

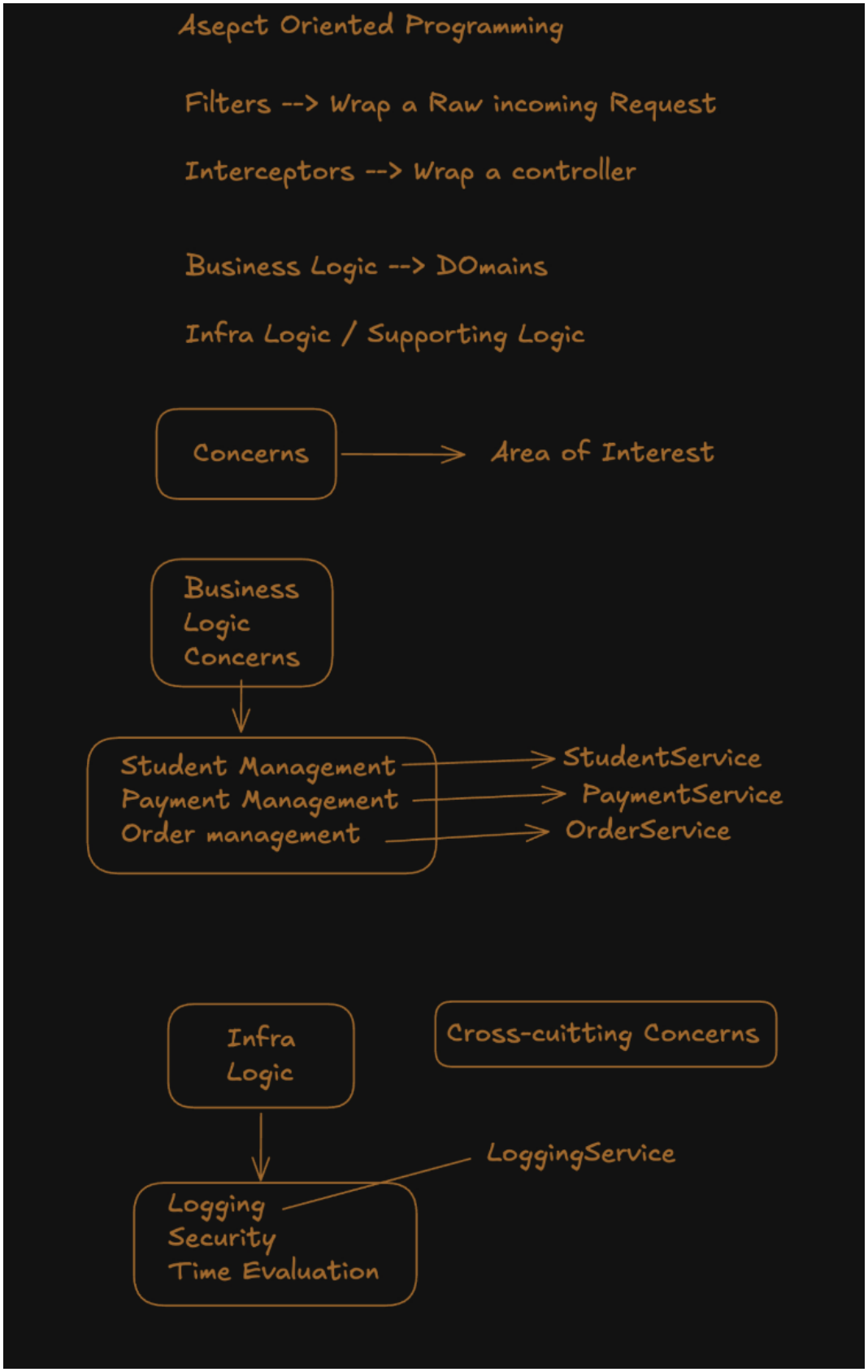
Infrastructure/supporting logic supports the business operation, such as logging, security/permission checks, audit recording, execution-time measurement, caching, and common exception-related handling. The lecture describes this as boilerplate that may recur across many applications.

3. Concerns and Cross-Cutting Concerns

A **concern** is an area of interest in an application. Student management, payment management, and order management are business concerns. Their logic is usually grouped into their respective services.

Cross-cutting concerns, such as logging, security, and timing, are needed in many classes and methods. They span across the business code rather than belonging to only one domain service.

4. Diagram 1 — Business Concerns and Infrastructure Concerns


Asepct Oriented Programming
Filters --> Wrap a Raw incoming Request
Interceptors --> Wrap a controller
Business Logic --> DOmains
Infra Logic / Supporting Logic
Concerns
Area of Interest
Business Logic Concerns
Student Management
Payment Management
Order management
StudentService
PaymentService
OrderService
Infra Logic
Cross-cuitting Concerns
LoggingService
Logging
Security
Time Evaluation

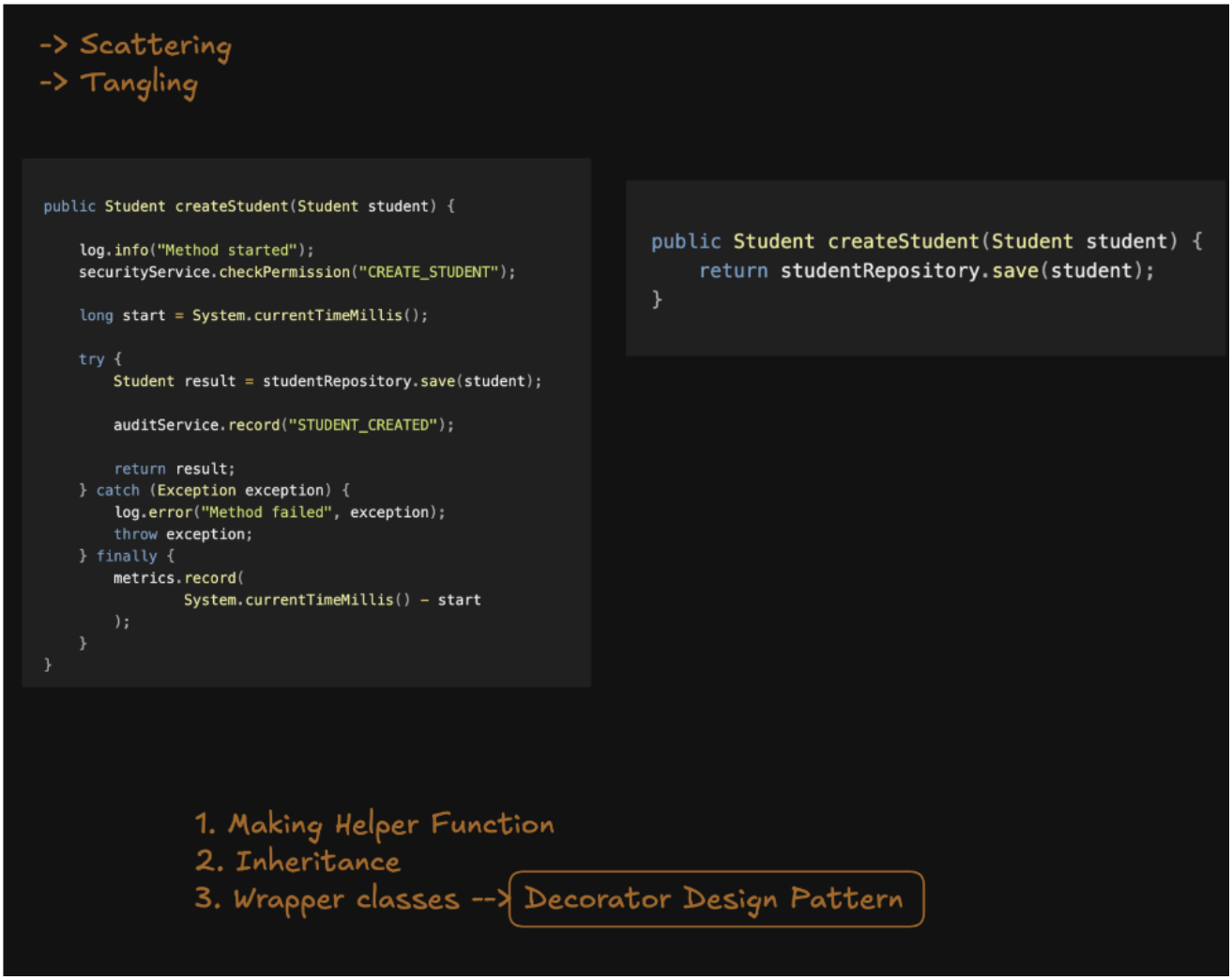
5. Scattering and Tangling

Scattering happens when the same supporting concern is spread across many classes. For example, logging calls may appear in StudentService, PaymentService, OrderService, and other services.

Tangling happens when one business method contains multiple kinds of logic mixed together. A createStudent() method may contain permission checks, logging, timing, auditing, exception handling, and the actual repository save.

These two problems make methods harder to read and maintain. Developers must understand infrastructure details before reaching the small amount of business logic the method is supposed to perform.

6. Diagram 2 — Scattering, Tangling, and the Clean Business Method



The left code sample mixes infrastructure logic with the business operation. The right sample shows the desired clean method: it performs the business operation (saving the student) without embedding logging, security, auditing, or timing code inside it.

7. Why Simple Helper Functions Do Not Fully Solve It

A utility/helper class can centralize logging code, but every method still has to call the helper at the correct places. Developers may forget a call, and start/end logging must also handle exceptions and different execution paths. The business method remains cluttered with infrastructure calls.

Inheritance has a similar limitation: a service can inherit helper behavior, but it still needs explicit calls to the parent methods. The infrastructure code remains mixed into the business method.

8. Decorator / Wrapper Approach

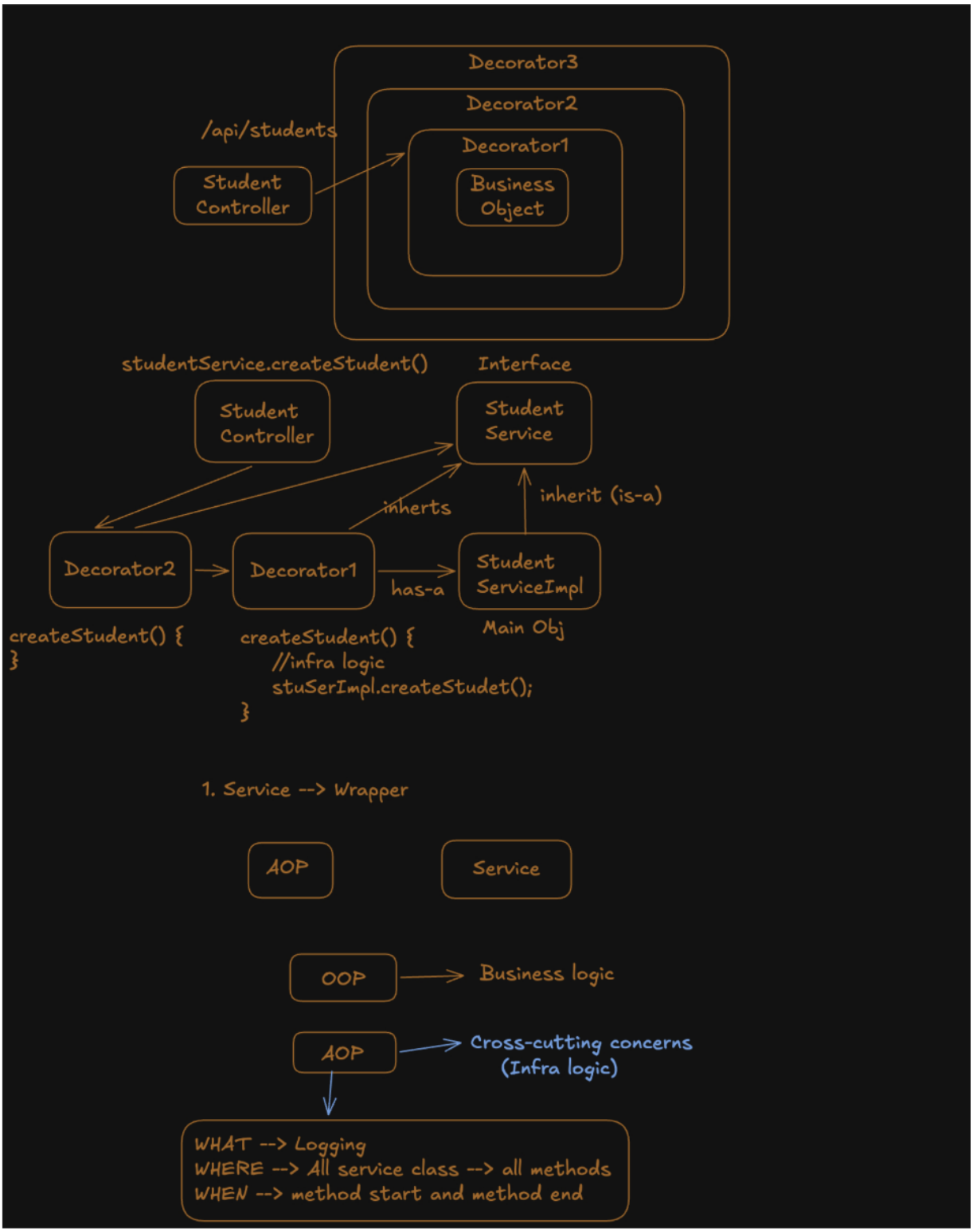
The lecture introduces wrapper classes through the Decorator design pattern. The business object exposes an interface; a decorator implements the same interface and holds a reference to the real implementation. The caller receives the decorator as if it were the business service.

When the caller invokes createStudent(), the decorator can run supporting logic first, delegate to the real StudentServiceImpl to perform the business operation, and then run supporting logic afterward.

Request flow (top to bottom)

```
StudentController calls the service interface
↓
Decorator receives createStudent()
↓
Decorator runs supporting logic (for example, log start)
↓
Decorator delegates to StudentServiceImpl
↓
StudentServiceImpl performs the business logic
↓
Control returns to the decorator
↓
Decorator runs supporting logic (for example, log finish)
↓
Result returns to StudentController
```

9. Diagram 3 — Nested Decorators and AOP


Decorator3
Decorator2
Decorator1
Business Object
/api/students
Student Controller
studentService.createStudent()
Interface
Student Controller
Student Service
inherits
inherit (is-a)
Decorator2
Decorator1
has-a
Student ServiceImpl
Main Obj
createStudent() {
}
createStudent() {
//infra logic
stuSerImpl.createStudet();
}
1. Service --> Wrapper
AOP
Service
OOP
Business logic
AOP
Cross-cutting concerns
(Infra logic)
WHAT --> Logging
WHERE --> All service class --> all methods
WHEN --> method start and method end

10. What the Decorator Diagram Explains

The outer wrappers represent multiple supporting concerns around one business object. For example, one wrapper may handle logging, another execution-time measurement, and another authentication or authorization. The wrappers can call each other in sequence before reaching the real implementation.

The important relationships in the diagram are:

- **Is-a:** the decorator implements the same service interface, so it can be used wherever the service is expected.
- **Has-a:** the decorator holds a reference to the next decorator or the real service implementation and delegates the business call to it.
- **Business object:** the actual service implementation contains the domain/business logic.
- **Decorator:** adds supporting behavior before and/or after delegation.

11. OOP and AOP

OOP structures the application's business logic and domain objects. **AOP** addresses cross-cutting concerns such as logging and execution-time measurement that span multiple methods/classes.

12. Key Takeaways

1. Keep business logic focused on the domain operation.
2. Logging, security checks, audit records, and timing are examples of supporting concerns.
3. Repeating infrastructure logic across classes causes scattering.
4. Mixing infrastructure logic into business methods causes tangling.
5. Helper methods and inheritance do not remove the need to call supporting logic explicitly.
6. A decorator/wrapper can surround the business service while keeping the caller dependent on the same interface.
7. AOP provides a framework-level way to apply such cross-cutting behavior without manually writing those calls in every business method.

13. Interview Questions

Q1. What problem does AOP solve?
A. It separates cross-cutting concerns from business logic, reducing repeated supporting code.

Q2. What is a cross-cutting concern?
A. A concern such as logging or timing that must be applied across many classes or methods.

Q3. Explain scattering vs tangling.
A. Scattering spreads the same concern across many classes; tangling mixes multiple concerns inside one method.

Q4. Why is a helper class alone insufficient?
A. Methods still need explicit calls to the helper, so the business code remains coupled to supporting logic.

Q5. How does the decorator idea help?
A. It wraps an implementation behind the same interface, adds supporting behavior, and delegates to the real business object.